

Chapitre III :

Gestion des exceptions



- ▶ Un programme PL/SQL peut provoquer des erreurs telles que l'accès à une donnée non existante dans la base, l'affectation de plusieurs données à une seule variable, une division par zéro ou une valeur erronée, etc.
- ▶ Ces erreurs sont appelées en PL/SQL des exceptions .
- ▶ Une exception peut être :
 - Une erreur générée par le système: mémoire insuffisante, index dupliqué
 - Une erreur induite par une action de l'utilisateur
 - Un avertissement (warnings) de l'application à l'utilisateur
- ▶ PL/SQL détecte et traite les erreurs grâce à une architecture de gestionnaires d'exceptions. Ce mécanisme permet de séparer proprement le code de traitement d'erreur des ordres exécutables (modèle de programmation par événements).



► Ils sont gérés par le gestionnaire des exceptions permettant au développeur de planifier leur gestion et d'abandonner ou de continuer le traitement approprié que le système doit exécuter en présence d'une erreur.

► Possibilité de continuer l'exécution si l'erreur n'est pas suffisamment importante pour produire la terminaison de la procédure

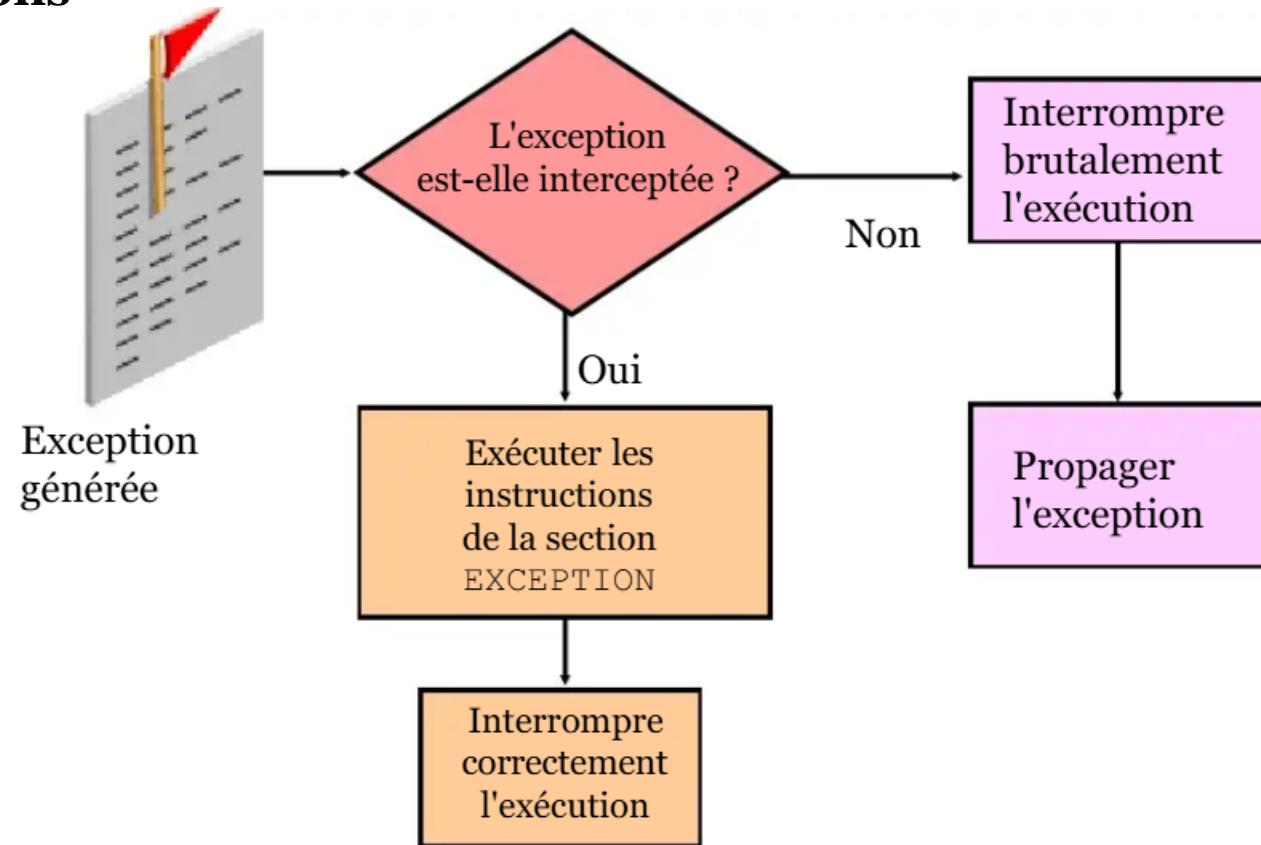
► On distingue deux types d'erreurs:

► **Les exceptions internes** Exception déclenchée implicitement:
→ Exception **prédéfinie** du serveur Oracle
→ Exception **non prédéfinie** du serveur Oracle

► **Les exceptions utilisateurs** Exception déclenchée explicitement:
→ Exception **définie** par l'utilisateur



► Traitement des exceptions





- ▶ Les exceptions internes sont caractérisées par des conditions d'erreurs connues par le serveur Oracle qu'il peut détecter et signaler implicitement.
- ▶ Toutes les exceptions internes répertoriées par Oracle sont libellées et codées. Le code est de la forme **ORA-dddd** (où d est un digit).
- ▶ Certaines exceptions internes sont en plus nommées par Oracle ; on dit qu'elles sont prédéfinies.
- ▶ Le nom permettra de les capter par un gestionnaire approprié.
- ▶ D'autres sont anonymes ; dites non prédéfinies, peuvent être nommées par le développeur. (procédure **PRAGMA EXCEPTION_INIT** du package **DBMS_STANDARD**).



Nom de l'exception	Code Oracle	Situation
DUP_VAL_ON_INDEX	ORA-00001	Lorsqu'une commande SQL tente de placer une valeur dupliquée dans une colonne définie avec un index unique.
VALUE_ERROR	ORA-06502	En cas de troncature ou d'erreur de conversion.
INVALID_NUMBER	ORA-01722	Lorsqu'une commande SQL spécifie une chaîne représentant un nombre invalide c'est-à-dire ne pouvant pas être convertie en une valeur numérique.
NO_DATA_FOUND	ORA-01403	Lorsqu'un curseur implicite d'interrogation n'identifie aucun n-uplet.
TOO_MANY_ROWS	ORA-01422	Lorsqu'un curseur implicite d'interrogation identifie plus qu'un n-uplet à retourner.
ZERO_DIVIDE	ORA-01476	Provoquée par une tentative de division par zéro.
INVALID_CURSOR	ORA-01722	Lorsqu'un curseur fermé est référencé dans une opération de lecture (FETCH), de fermeture (CLOSE) ou via un des attributs (%FOUND, %NOTFOUND, %ROWCOUNT).



Exceptions prédéfinies

► Exemple 1:

```
Var a number;  
BEGIN  
Select department_name into :a from departments;  
END;  
/
```

correction

```
BEGIN  
Select department_name into :a from departments;  
  
Exception  
When too_many_rows then dbms_output.put_line('plusieurs  
lignes');  
END;  
/
```

```
ERREUR ó la ligne 1 :  
ORA-01422: l'extraction exacte ramène plus que le nombre de lignes demandé  
ORA-06512: ó ligne 2
```

Les exceptions internes

Exceptions prédéfinies

Gestion des exceptions



► Exemple 2:

DECLARE

b departments.department_name%type;

A number :=1;

BEGIN

Select department_name into b from departments
where department_id=a;

Dbms_output.put_line(b);

END;

/

```
ERREUR à la ligne 1 :  
ORA-01403: aucune donnée trouvée  
ORA-06512: à ligne 3
```

correction

DECLARE

b departments.department_name%type; A number :=1;

BEGIN

Select department_name into b from departments where
department_id=a;

Dbms_output.put_line(b);

Exception

When no_data_found then dbms_output.put_line('pas de données');

END;

/



Exceptions prédéfinies

Chaque gestion d'exception consiste à spécifier son nom d'erreur après la clause **WHEN** et la séquence de commande à exécuter après le mot **THEN**.

► Exemple 3:

```
DECLARE
    Vpu article.pu%type ;
BEGIN
    select pu into vpu from article;
EXCEPTION
    --traiter le cas où le select retourne plusieurs lignes
    When Too_many_rows Then dbms_output.put_line('La requête ramène
    plusieurs lignes');
    --traiter le cas où le select ne retourne aucune ligne
    When No_data_found Then dbms_output.put_line('Pas d''articles dans la
    BD');
END;
```



Exceptions non prédéfinies

Nous pouvons associer un code erreur Oracle à des variables exception à l'aide du mot clé **PRAGMA EXCEPTION_INIT**, dans la section **DECLARE**

```
DECLARE  
  Nom_exception EXCEPTION ;  
  PRAGMA EXCEPTION_INIT(nom_exception, -  
code_error_oracle);
```

Exemple: Lorsque l'on tente d'insérer une chaîne de caractères dans une variable que sa déclaration ne le permet, Oracle déclenche une erreur **-6502**. Nous allons « nommer » cette erreur **trop_long** et l'intercepter dans la section **Exception**.



Exceptions non prédéfinies

Exemple: Lorsque l'on tente d'insérer une chaîne de caractères dans une variable que sa déclaration ne le permet, Oracle déclenche une erreur **-6502**. Nous allons « nommer » cette erreur **trop_long** et l'intercepter dans la section **Exception**.

```
Declare
  Chaine varchar2(10) ;
  trop_long exception ;
  pragma exception_init(trop_long, -6502 ) ;
Begin
  Chaine := rpad('-', 30) ;
Exception
  when trop_long then
    dbms_output.put_line( 'Chaîne de caractères trop longue' ) ;
End ;
```



Exceptions génériques: OTHERS

- L'exception prédéfinie **OTHERS** est une exception générique capable de capter toute erreur pour laquelle aucun gestionnaire n'a été prévu dans le bloc. Elle est très utile pour traiter les erreurs imprévisibles.
- **SQLCODE** : Renvoie un entier représentant le n° de l'erreur interne qui a provoqué l'exception et zéro s'il n'y a pas d'erreur.
- **SQLERRM** : Renvoie un texte, composé du numéro de l'erreur et d'un message complet associé à l'exception rencontrée, de la forme : ORA-ddddd : texte explicatif.

► Exemple 1:

```
DECLARE
    n number := 20;
BEGIN
    n := n/0 ;
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line( 'CODE ERREUR : ' || to_char(SQLCODE) || ' LIBELLEE ERREUR : ' || SQLERRM);
END;
```



► La gestion des exceptions utilisateur peut se faire dans un bloc PL/SQL en effectuant les opérations suivantes:

- Nommer l'anomalie (de type **EXCEPTION**) dans la section **DECLARE**.
- Déterminer l'erreur (condition d'apparition) et passer la main au traitement approprié par la commande **RAISE**.
- Effectuer le traitement défini dans la partie exception du bloc.



DECLARE

```
--declaration de l'exception  
nom_anomalie EXCEPTION ;
```

BEGIN

```
--définir la condition d'apparition de l'anomalie  
IF(condition) then  
    RAISE nom_anomalie ;  
END IF;
```

EXCEPTION

```
--effectuer le traitement de l'anomalie  
WHEN nom_anomalie THEN traitement;  
END;
```

1 Déclarer l'exception

2 Évoquer l'exception

3 Définir l'exception



Exceptions utilisateurs

Exemple:

```
Declare
  Vpu article.pu%type ;
  Pu_zero exception;
Begin
  select pu into vpu from article where nart=100;
  if vpu=0
    then raise pu_zero;
  end if;
Exception
  --traitement de l'exception externe pu_zero
  when pu_zero THEN
    delete from Article where nart=100;
    dbms_output.put_line('Le produit 100 est supprimé car son PU égale à 0 ');

  --traiter le cas où le select ne retourne aucune ligne
  when No_data_found THEN dbms_output.put_line('Table article vide');
End;
```



- ▶ Les exceptions utilisateurs peuvent être créés dans la partie déclaration d'un sous programme
 - ▶ Ils ne sont accessibles qu'à l'intérieur de ce sous-programme
- ▶ Nous pouvons lever à la fois des exceptions définies par l'utilisateur et/ou définies par le système
- ▶ Les règles de visibilité des exceptions sont les mêmes que celles des variables



- **RAISE_APPLICATION_ERROR** est une procédure du package **DBMS_STANDARD**, utile pour produire des messages d'erreur applicatifs spécifiques à l'environnement appelant.
- L'exécution de cette commande reproduit l'erreur et annule la transaction en cours. Elle est utilisée dans les sections **BEGIN** et **EXCEPTION**.

```
RAISE_APPLICATION_ERROR(num_erreur,msg_erreur[, {TRUE|FALSE}]);
```

- **num_erreur** : représente un entier négatif compris entre -20000 et -20999
- **msg_erreur** : représente le texte du message d'une longueur maximum de 2048 octets
- **TRUE** : indique que l'erreur est ajoutée à la pile des erreurs précédentes.
- **FALSE** : indique que l'erreur remplace toutes les erreurs précédentes

Envoi de message d'erreur

```
Declare
Total number;
PuExc exception;
QtExc exception;
PRAGMA EXCEPTION_INIT(PuExc, -20001);
PRAGMA EXCEPTION_INIT(QtExc, -20002);
Begin
Total:=calculTotalTTC(0,10,19);
/* la présence d'une exception entraine l'execution du
traitement approprié puis interruption de l'execution du
bloc*/
dbms_output.put_line('Total TTC:' || Total);

Exception
When PuExc then
    Dbms_output.put_line ('***Vérifier le PU***');
When QtExc then
    Dbms_output.put_line ('***Vérifier la Quantité***');
When others then
    Dbms_output.put_line (SQLCODE || ':' || SQLERRM);
End;
```

```
Create or replace function calculTotalTTC (PU number, QTE
number, TVA number) return number
Is

Begin
if (PU <=0) then
    RAISE_APPLICATION_ERROR(-20001,'Veuillez vérifier le prix unitaire', true);
End if;

/*l'ajout du paramètre true rend obligatoire le traitement de l'exception -
20001 dans le bloc appellant. Pour cela il va falloir nommer cette exception
avec la procedure pragma exception_int, et ce, afin de pouvoir definir un
traitement approprié dans le gestionnaire d'exception. Sinon, on peut
utiliser l'exception générique OTHERS*/

if (QTE <=0) then
    RAISE_APPLICATION_ERROR(-20002,'Veuillez vérifier l
a quantité', true);
End if;

Return (PU*QTE*(1+TVA/100));

End;
```

Envoi de message d'erreur

```
Declare
Total number;
codeERR number;
msgERR varchar(100);

PuExc exception;
QtExc exception;
PRAGMA EXCEPTION_INIT(PuExc, -20001);
PRAGMA EXCEPTION_INIT(QtExc, -20002);
Begin
Total:=calculTotalTTC(0,10,19);
/* la présence d'une exception entraine l'execution du
traitement approprié puis interruption de l'execution du bloc*/

dbms_output.put_line('Total TTC:' || Total);

Exception
When PuExc then
    Dbms_output.put_line ('***Vérifier le PU***');
When QtExc then
    Dbms_output.put_line ('***Vérifier la Quantité***');
When others then
    codeERR := SQLCODE;
    msgERR := SQLERRM;
    Dbms_output.put_line (codeERR || ':' || msgERR);
End;
```

Create or replace function calculTotalTTC (PU number, QTE number, TVA number)
return number

Is

–PUNegatif exception;

–QTNegative exception;

Begin

if (PU <=0) then

RAISE_APPLICATION_ERROR(-20001,'Veuillez vérifier le prix unitaire',

true);

End if;

/*l'ajout du paramètre true rend obligatoire le traitement de l'exception -20001 dans le bloc appellant. Pour cela il va falloir nommer cette exception avec la procedure pragma exception_int, et ce, afin de pouvoir definir un traitement approprié dans le gestionnaire d'exception. Sinon, on peut utiliser l'exception générique OTHERS*/

if (QTE <=0) then

RAISE_APPLICATION_ERROR(-20002,'Veuillez vérifier la quantité', true);

End if;

Return (PU*QTE*(1+TVA/100));

–Exception

–when PUNegatif THEN

– RAISE_APPLICATION_ERROR(-20001,'Veuillez vérifier le prix unitaire', true);

–when QTNegative THEN

–RAISE_APPLICATION_ERROR(-20002,'Veuillez vérifier la quantité',true);

End;